Lab W3 D5

Jose

Q1.

	A	B	C	D	E	F	G	H	I
A	0	22	9	12	0	0	0	0	0
B	22	0	35	0	0	36	0	34	0
C	9	35	0	4	65	42	0	0	0
D	12	0	4	0	33	0	0	0	30
E	0	0	65	33	0	18	23	0	0
F	0	36	42	0	18	0	39	24	0
G	0	0	0	0	23	39	0	25	21
H	0	34	0	0	0	24	25	0	19
I	0	0	0	30	0	0	21	19	0

Q2. Dijkstra's Algorithm:

vertex	distance	Path	visited	Unvisited
A	0	-	A	BCDEFGHI
C	9	AC	A	BCDEFGHI
D	12	AD	A	BCDEFGHI
B	22	AB	A	BCDEFGHI
~~D~~	~~13~~	~~ACD~~	AC	BDEFGHI
~~B~~	~~44~~	~~ACB~~	AC	BDEFGHI
F	51	ACF	AC	BDEFGHI
~~E~~	~~74~~	~~ACE~~	AC	BEFGHI
I	42	ADI	ACD	BEFGHI
E	45	ADE	ACD	EFGHI
H	56	ABH	ACDB	EFGHI
~~F~~	~~58~~	~~ABF~~	ACDB	EFGH
~~H~~	~~61~~	~~ADIH~~	ACDBI	EFGH
G	63	ADIG	ACDBI	FGH
~~F~~	~~63~~	~~ADEF~~	ACDBIE	FGH
~~G~~	~~68~~	~~ADEG~~	ACDBIE	GH
~~H~~	~~75~~	~~ACFH~~	ACDBIEF	GH
~~G~~	~~90~~	~~ACFG~~	ACDBIEF	G
~~G~~	~~81~~	~~ABHG~~	ACDBIEFH	

Q3. Time complexity is $O(m \log n)$

Q4. Kruskal's Minimum Spanning Tree

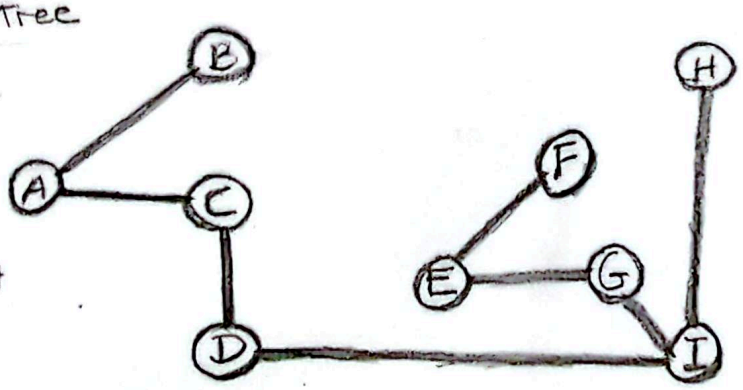
- List of edges sorted by weight.

✓ (C, D) 4
✓ (A, C) 9
✗ (A, D) 12
✓ (E, F) 18
✓ (H, I) 19
✓ (G, I) 21
✓ (A, B) 22
✓ (E, G) 23
✗ (F, H) 24
✗ (H, G) 25
✓ (D, I) 30
✗ (D, E) 33
✗ (B, H) 34
✗ (B, C) 35
✗ (B, F) 36
✗ (F, G) 39
✗ (C, F) 42
✗ (C, E) 65

Initialize: {A}, {B}, {C}, {D}, {E}, {F}, {G}, {H}, {I}

- find(C) ≠ find(D). Union(C, D)
  {A}, {B}, {C, D}, {E}, {F}, {G}, {H}, {I}
- find(A) ≠ find(C). Union(A, C)
  {A, C, D}, {B}, {E}, {F}, {G}, {H}, {I}
- find(A) == find(D). Delete(A, D)
- find(E) ≠ find(F). Union(E, F)
  {A, C, D}, {B}, {E, F}, {G}, {H}, {I}
- find(H) ≠ find(I). Union(H, I)
  {A, C, D}, {B}, {E, F}, {G}, {H, I}
- find(G) ≠ find(I). Union(G, I)
  {A, C, D}, {B}, {E, F}, {G, H, I}
- find(A) ≠ find(B). Union(A, B)
  {A, B, C, D}, {E, F}, {G, H, I}
- find(E) ≠ find(G). Union(E, G)
  {A, B, C, D}, {E, F, G, H, I}
- find(F) == find(H). Delete(F, H)
- find(H) == find(G). Delete(H, G)
- find(D) ≠ find(I). Union(D, I)
  {A, B, C, D, E, F, G, H, I}
- find(D) == find(E). DELETE(D, E) ... Same for remaining edges.

✓ Spanning Tree



Total weight is 146

Q5. Time complexity is O(m log n)

W3D5

Q6. Adjacency Matrix

	P	Q	R	S	T	U
P	0	1	0	6	7	0
Q	0	0	1	4	0	0
R	0	0	0	2	0	1
S	0	0	0	0	3	2
T	0	0	0	0	0	2
U	0	0	0	0	0	0

Q7. Dijkstra's Dynamic Programming Approach

Topological ordering : PQRSTU

Vertex	Distance	Path
P	0	—
Q	1	PQ
R	2	PQR
S	5	PQS
T	7	PT
U	3	PQRU ✓

⇒ Shortest Path from P to U
: {(P,Q),(Q,R),(R,U)}
⇒ 1+1+1 = 3

Q8. Time complexity is $O(n+m)$

Q9. Yes, It is possible to use Dijkstra's algorithm to find the shortest path from P to U.

Q10 Dijkstra's algorithm

	Vertex	Distance	Path
	P	0	—
	Q	1	P Q
X	~~S~~	~~6~~	~~P S~~
	T	7	P T
	R	2	P Q R
	S	5	P Q S
X	~~U~~	~~7~~	~~P Q S U~~
X	~~T~~	~~8~~	~~P Q S T~~
X	~~U~~	~~9~~	~~P T U~~
	U	3	P Q R U ✓

the shortest Path from P to U applying Dijkstra's algorithm is PQRU

$\Rightarrow$ Path $\{(P,U)\} = \{(P,Q), (Q,R), (R,U)\}$

$= 1 + 1 + 1$

$= 3$